

Java2 Day01 핵심 정리

인터페이스 활용, 객체 정렬, 제네릭, 와일드카드를 처음 배우는 사람도 이해할 수 있게 정리

주제: 인터페이스, 객체 정렬, 제네릭, 와일드카드

대상 독자: Java 기본 문법을 배운 뒤 Java2 수업을 복습하는 초보자

작성 기준 자료: Day1 TXT, Day1 원본 PDF, java2prj day1 소스 코드

생성일: 2026-05-21

인터페이스

클래스가 반드시 지켜야 할 기능 약속입니다.

객체 정렬

객체를 어떤 기준으로 앞뒤 비교할지 정하는 작업입니다.

제네릭

타입을 나중에 정하게 해서 재사용성을 높이는 문법입니다.

와일드카드

메서드 매개변수에서 받을 수 있는 제네릭 타입 범위를 넓히는 문법입니다.

0. 목차와 학습 로드맵

단원	학습 내용
1	Java2 Day01 전체 개요
2	인터페이스로 라이브러리 만들기
3	Comparable과 Comparator로 객체 정렬하기
4	인터페이스 default/static 메서드
5~7	제네릭 클래스, 제네릭 메서드, 와일드카드
8	실습 정리
9	연습 문제와 정답 해설

1. 먼저 인터페이스가 왜 필요한지 이해합니다.
2. 객체 정렬에서 비교 기준을 어디에 두는지 확인합니다.
3. 제네릭을 "타입을 나중에 정하는 문법"으로 이해합니다.
4. 와일드카드는 메서드 매개변수를 유연하게 만드는 문법으로 정리합니다.
5. 마지막에 연습 문제로 개념을 확인합니다.

원본 자료	확인 내용	반영 방식
자바2_1차시.txt	전체 내용 확인	학습 목표와 실습 항목 반영
Day1 원본 PDF 7개	파일 존재, PDF 헤더, 페이지 구조 확인	PDF 주제와 TXT/소스 코드 기준으로 개념 재구성
workspace/java2prj/src/day1	133개 파일 구조와 대표 예제 확인	인터페이스, 정렬, 제네릭, 와일드카드 코드 예제로 반영

확인 제한: 로컬 환경에 PDF 텍스트 추출 도구가 없어 원본 PDF의 세부 본문 텍스트는 완전 추출하지 못했습니다. 이 자료는 TXT 전체 내용과 실제 소스 코드 확인 내용을 중심으로 재구성했습니다.

1. Day01에서 배우는 Java 문법

Day01은 Java2의 출발점입니다. 단순히 클래스를 만들고 실행하는 수준에서 벗어나 **여러 클래스가 같은 규칙을 따르게 만들고, 객체를 정렬하고, 타입을 더 안전하게 다루는 방법**을 배웁니다.

개념	쉬운 뜻	왜 필요한가
interface	기능 목록만 적어 둔 약속	서로 다른 클래스도 같은 방식으로 다루기 위해
Comparable	객체 자신의 기본 정렬 기준	<code>Collections.sort(list)</code> 만으로 정렬하기 위해
Comparator	외부에서 따로 주는 정렬 기준	이름순, 점수순, 반순처럼 기준을 바꾸기 위해
Generic	타입을 변수처럼 쓰는 문법	형변환 오류를 줄이고 코드를 재사용하기 위해
Wildcard	?로 타입 범위를 표현	메서드가 더 다양한 제네릭 타입을 받게 하기 위해

핵심 흐름: Day01은 "정해진 타입 하나만 쓰는 코드"에서 "여러 타입과 여러 구현체를 안전하게 다루는 코드"로 넘어가는 수업입니다.

2. 인터페이스로 라이브러리 만들기

인터페이스는 "이 기능은 반드시 있어야 한다"는 약속입니다. 라이브러리를 만들 때 인터페이스를 매개변수로 받으면, 라이브러리는 구체적인 클래스 이름을 몰라도 됩니다.

```
public interface MyRunnable {
    void run();
}
```

```
class MyLib {
    public void codeRun(MyRunnable job) {
        job.run();
    }
}
```

```
MyLib lib = new MyLib();
lib.codeRun(new A());
lib.codeRun(new B());
lib.codeRun(new C());
```

MyLib는 A, B, C를 직접 알 필요가 없습니다. 단지 MyRunnable을 구현한 객체라면 run()을 호출할 수 있습니다.

비유: 콘센트는 노트북 회사 이름을 몰라도 됩니다. 플러그 규격만 맞으면 전기를 공급합니다. 인터페이스도 같은 역할입니다.

3. 객체 정렬: Comparable과 Comparator

숫자나 문자열은 Java가 기본 정렬 방법을 알고 있습니다. 하지만 Student, Webtoon, Moving 같은 객체는 어떤 기준으로 정렬해야 할지 Java가 모릅니다. 그래서 정렬 기준을 알려줘야 합니다.

3-1. Comparable: 객체 안에 기본 정렬 기준 넣기

```
class Student implements Comparable<Student> {
    String name;
    int ban;
    int totalScore;

    public int compareTo(Student other) {
        return other.totalScore - this.totalScore; // 총점 내림차순
    }
}
```

```
ArrayList<Student> list = new ArrayList<>();
Collections.sort(list);
```

Comparable은 "이 객체는 기본적으로 이렇게 정렬된다"는 기준입니다. 위 예제에서는 점수가 높은 학생이 앞에 오도록 만들었습니다.

3-2. Comparator: 정렬 기준을 밖에서 주기

```
Collections.sort(list, Comparator.comparing(s -> s.getName()));
Collections.sort(list, Comparator.comparing(s -> s.getBan()));
Collections.sort(list, Comparator.comparing(Student::getName).reversed());
```

Comparator는 외부에서 임시로 정렬 기준을 주는 방식입니다. 같은 학생 리스트라도 이름순, 반순, 점수순처럼 필요할 때마다 기준을 바꿀 수 있습니다.

구분	Comparable	Comparator
위치	객체 클래스 내부에 작성	정렬하는 곳에서 작성
메서드	compareTo()	compare() 또는 Comparator.comparing()
용도	기본 정렬 기준	상황별 정렬 기준
예	학생은 기본적으로 총점순	오늘은 이름순, 내일은 반순

정렬의 핵심: 객체 정렬은 "객체끼리 비교했을 때 누가 앞에 와야 하는가"를 코드로 표현하는 것입니다.

4. 인터페이스의 default와 static 메서드

원래 인터페이스는 추상 메서드만 가진다고 배웠지만, Java 8부터는 default와 static 메서드도 가질 수 있습니다.

```
public interface 한식요리가능한 {
    void 구절판만들기();
    void 불고기전골만들기();

    default void 김치만두만들기() {
        System.out.println("김치만두 만들기");
    }

    static void 김치만두만들기s() {
        System.out.println("공통 김치만두 레시피");
    }
}
```

구분	default 메서드	static 메서드
호출 방법	구현 객체로 호출	인터페이스 이름으로 호출
오버라이드	가능	불가능
의미	구현 클래스에 기본 기능 제공	인터페이스에 묶인 공통 도구 기능

5. 제네릭 클래스

제네릭은 타입을 나중에 정할 수 있게 하는 문법입니다. Box<Integer>라고 쓰면 정수 전용 박스가 되고, Box<String>이라고 쓰면 문자열 전용 박스가 됩니다.

```
public class Box<T> {
    Object[] list = new Object[10];
    int index = 0;

    public void add(T item) {
        list[index] = item;
        index++;
    }

    public T get(int i) {
        return (T) list[i];
    }

    public int size() {
        return index;
    }
}
```

```
Box<Integer> intBox = new Box<>();
intBox.add(1);
intBox.add(2);

Box<String> strBox = new Box<>();
strBox.add("hi");
strBox.add("hello");
```

T는 정해진 타입 이름이 아닙니다. "나중에 사용하는 사람이 타입을 정해 주세요"라는 표시입니다.

6. 제네릭 메서드

제네릭 클래스는 클래스 전체가 타입을 받습니다. 제네릭 메서드는 **메서드 하나만** 타입을 받습니다.

```
public static <T> void print(T value) {
    System.out.println(value);
}

print(5);
print("hello 제네릭 메서드");
```

<T>는 이 메서드 안에서 사용할 타입 이름을 선언하는 부분입니다. 호출할 때 Java가 대부분 타입을 자동으로 추론합니다.

7. 와일드카드

와일드카드는 ?로 표현합니다. 주로 메서드 매개변수에서 "정확히 한 타입만 받는 게 아니라, 어느 정도 범위의 타입을 받고 싶다"는 의미로 사용합니다.

```
List<? extends Animal> animals = new ArrayList<Dog>();
Animal a = animals.get(0); // 읽기는 Animal로 가능
// animals.add(new Dog()); // 추가는 막힘
```

```
List<? super Animal> animals2 = new ArrayList<Object>();
animals2.add(new Dog());
animals2.add(new Cat());
animals2.add(new Animal());
Object obj = animals2.get(0);
```

표현	뜻	기억법
?	어떤 타입이든 가능	모름
? extends T	T 또는 T의 자식 타입	읽기에 유리
? super T	T 또는 T의 부모 타입	쓰기에 유리

extends는 읽기, super는 쓰기라고 먼저 외우면 좋습니다. 정확한 이유는 “실제로 어떤 타입의 리스트인지 컴파일러가 확신할 수 있는가”와 관련이 있습니다.

8. 실습 정리

- 라이브러리 실습: 인터페이스를 매개변수로 받아 원하는 코드를 실행하게 만들기
- 정렬 실습: ArrayList<Webtoon> 또는 객체 배열을 이름순, 점수순 등으로 정렬하기
- 제네릭 실습: 문자열, 정수, 실수 배열에서 특정 값의 위치를 찾는 메서드를 제네릭으로 일반화하기
- 오늘의 요리사: Cook 상속과 한식/중식/일식 인터페이스 구현으로 그룹을 나누고 랜덤 선택하기

9. 연습 문제와 정답 해설

문제

- 인터페이스를 매개변수로 받는 라이브러리 코드의 장점은 무엇인가?
- Comparable과 Comparator의 차이를 설명하시오.
- Box<T>에서 T는 무엇을 의미하는가?
- ? extends Animal과 ? super Animal의 차이를 설명하시오.

정답 및 해설

번호	정답 해설
1	구체 클래스 이름을 몰라도 약속된 메서드를 호출할 수 있어 코드가 유연해집니다.
2	Comparable은 객체 내부의 기본 정렬 기준이고, Comparator는 정렬할 때 외부에서 주는 기준입니다.
3	T는 나중에 사용자가 정하는 타입 자리입니다. 예: Box<String>, Box<Integer>.
4	extends는 읽기에 유리하고, super는 쓰기에 유리합니다.

10. 원본 출처와 확인 불가 항목

항목	이유	필요한 추가 자료
Day1 원본 PDF 세부 본문 전체 추출	PDF 텍스트 추출 도구 없음	텍스트 추출본 또는 OCR 결과
코드 실행 결과	이번 자료 제작에서는 실행 검증하지 않음	Eclipse 또는 javac 실행 환경 확인

11. 마지막 핵심

주제	반드시 기억할 말
인터페이스	구현 클래스가 지켜야 할 약속 이다.
Comparable	객체 안에 들어가는 기본 정렬 기준이다.
Comparator	정렬할 때 외부에서 주는 기준이다.
제네릭	타입을 나중에 정해서 안전성과 재사용성을 높인다.
와일드카드	메서드 매개변수에서 제네릭 타입 범위를 유연하게 만든다.